
State Transition Matrix / Status Algebra Layer v0.1

Formal transition discipline for evidentiary, quarantine, challenge, and runtime status in the Triad Stack.

STATUS

Concept Draft

SCOPE

Formal state transition discipline for the Triad Stack and its evidence, challenge, quarantine, and reopenability mechanics

POSITION IN STACK

Follows Challenge / Review Protocol Layer; prepares direct bridge to implementation logic in ester-clean-code

Kotov Ivan

Bruxelles | 2026

1. Purpose

The stack now contains:

- L4 Glitch Map
- Research / Backward Node
- Cinematic Walkthrough Layer
- Triad Minimal Stack
- Graph Grammar / Schema Layer
- Graph Rendering Conventions / Visual Notation
- Witness Overlay / Evidence Notation Layer
- Challenge / Review Protocol Layer

That is enough to describe:

- what exists,
- how it is seen,
- what evidentiary standing it has,
- and how that standing may be contested.

It is not yet enough to guarantee one of the hardest things:

that the system changes state lawfully.

A mature architecture must not only say what a node *is*.

It must also say:

- how a node may become something else,
- under which preconditions,
- through which bounded transitions,
- and which transitions are categorically forbidden.

This is the role of the **State Transition Matrix / Status Algebra Layer**.

2. Core Principle

Status is not decoration.

Status is permissioned memory about what may happen next.

If status transitions are left implicit, the stack becomes vulnerable to:

- authority laundering,
- evidence inflation,
- cinematic override,
- silent reopening,
- and amnesia-by-reclassification.

Therefore every meaningful status must have:

1. defined semantics
 2. legal predecessor states
 3. legal successor states
 4. transition guards
 5. forbidden shortcuts
-

3. Status Domains

The stack does not contain one flat status system.

It contains **interacting status domains**.

3.1 Ontology Domain

What kind of object is this?

Examples:

- ExecutionNode
- GlitchNode
- ResearchNode
- BackwardNode
- CinematicSegment
- WitnessNode
- ReviewNode

Ontology usually does **not** change frequently.

3.2 Evidence Domain

What is the evidentiary standing of this object?

Examples:

- asserted
- observed
- witnessed
- signed
- settled
- expired
- cinematic_only

3.3 Quarantine / Reopenability Domain

May this object move toward renewed consideration?

Examples:

- quarantined
- reopenable
- reopened
- archived
- sealed

3.4 Challenge Domain

What is the dispute status?

Examples:

- none
- challenge_open
- queued
- under_review
- resolved_uphold
- resolved_modify
- resolved_split
- dismissed
- challenge_expired

3.5 Runtime Legibility Domain

How may the object be used operationally?

Examples:

- `display_only`
- `inspectable`
- `audit_usable`
- `research_usable`
- `runtime_non_executable`
- `runtime_blocked`

The system must not flatten these domains into one ambiguous label.

4. Algebraic Position

We define each graph object as carrying a tuple:

$S = (O, E, Q, C, R)$

Where:

- `O` = Ontology status
- `E` = Evidence status
- `Q` = Quarantine / reopenability status
- `C` = Challenge status
- `R` = Runtime legibility status

Example:

```
S(node_17) = (  
  ResearchNode,  
  witnessed,  
  quarantined,  
  challenge_open,  
  research_usable  
)
```

This means:

- the node is ontologically a research node,
 - it has witness-level evidence,
 - it remains quarantined,
 - a challenge is open,
 - and it may be used for research inspection but not execution.
-

5. Transition Philosophy

Transitions must obey three rules.

5.1 Non-Collapse Rule

A transition may alter one status domain without silently collapsing another.

Example:

- `challenge_open -> resolved_modify`

must not silently imply:

- `quarantined -> reopenable`

unless a second explicit transition is justified.

5.2 No Shortcut Rule

A later status may not be reached unless all required intermediate conditions have been satisfied.

Example:

- `asserted -> signed`

is forbidden.

5.3 Historical Preservation Rule

State transitions do not erase prior status meaning.

The graph remembers:

- what was first asserted,
- what later became witnessed,
- what was disputed,

- and what ultimately settled or expired.

6. Canonical Evidence Transition Matrix

6.1 Legal Evidence Transitions

From	To	Allowed	Guard
asserted	observed	yes	recorded event exists
observed	witnessed	yes	witness event attached
witnessed	signed	yes	signing discipline satisfied
signed	settled	yes	challenge window closed or resolved
observed	expired	yes	context drift / timeout
witnessed	expired	yes	trust or temporal expiry
signed	expired	yes	expiry condition met
cinematic_only	asserted	yes	explicit extraction from cinematic branch

6.2 Forbidden Evidence Transitions

From	To	Allowed	Reason
asserted	witnessed	no	missing observation stage
asserted	signed	no	missing witness discipline
cinematic_only	witnessed	no	aesthetics cannot become proof
expired	signed	no	expiry cannot be reversed by promotion

From	To	Allowed	Reason
settled	asserted	no	historical regression forbidden

7. Canonical Quarantine / Reopenability Matrix

7.1 Legal Transitions

From	To	Allowed	Guard
quarantined	reopenable	yes	new evidence or new valid resource/context
reopenable	reopened	yes	formal review approval + witness link
reopened	quarantined	yes	reopened path fails again
reopenable	archived	yes	relevance decays without activation
quarantined	sealed	yes	policy or ontology says never executable
reopened	archived	yes	branch completes without runtime promotion

7.2 Forbidden Transitions

From	To	Allowed	Reason
quarantined	reopened	no	missing reopenability step
quarantined	runtime_usable	no	quarantine cannot jump to action

From	To	Allowed	Reason
sealed	reopened	no*	requires exceptional institutional override
archived	reopened	no*	requires explicit revival protocol

* These may exist only under a distinct exceptional protocol, never as default transitions.

8. Canonical Challenge Matrix

8.1 Legal Transitions

From	To	Allowed	Guard
none	challenge_open	yes	eligible role + target + reason
challenge_open	queued	yes	accepted into review process
queued	under_review	yes	reviewer assigned
under_review	resolved_uphold	yes	review concludes original holds
under_review	resolved_modify	yes	review changes interpretation
under_review	resolved_split	yes	review requires branch split
challenge_open	dismissed	yes	insufficient procedural basis
challenge_open	challenge_expired	yes	deadline passes
resolved_*	archived	yes	review finalized

8.2 Forbidden Transitions

From	To	Allowed	Reason
challenge_open	resolved_modify	no	must pass review stage
none	resolved_uphold	no	no challenge existed
dismissed	resolved_modify	no	dismissed case cannot mutate silently
challenge_expired	under_review	no*	requires formal reopening of challenge

9. Runtime Legibility Matrix

This domain protects the stack from accidentally treating visible objects as executable.

9.1 Legal Transitions

From	To	Allowed	Guard
display_only	inspectable	yes	graph view or audit mode enabled
inspectable	research_usable	yes	research context + non-execution scope
research_usable	audit_usable	yes	witness sufficiency for audit use
runtime_non_execut	runtime_blocked	yes	explicit L4 failure or privilege stop

research_usable	runtime_non_execut	yes	status normalization under quarantine
-----------------	--------------------	-----	---

9.2 Forbidden Transitions

From	To	Allowed	Reason
display_only	runtime_usable	no	interface cannot create authority
cinematic_only	runtime_usable	no	representational branch not actionable
audit_usable	runtime_usable	no	audit relevance != execution permission
research_usable	runtime_usable	no	research utility != action authority

10. Composite Transition Examples

Because statuses are tuples, the most important transitions are composite.

10.1 L4 Collision

Before:

(ExecutionNode, observed, none, none, runtime_non_executable)

After:

(GlitchNode, witnessed, none, none, runtime_blocked)

Meaning:

- runtime path hit physical boundary,
- witnessable glitch now exists,
- execution is blocked,

- no challenge yet open.

10.2 Quarantine Formation

Before:

(GlitchNode, witnessed, none, none, runtime_blocked)

After:

(ResearchNode, witnessed, quarantined, none, research_usable)

Meaning:

- failure is preserved as research object,
- not action-authorized,
- visible and usable in bounded inquiry.

10.3 Review-Driven Reclassification

Before:

(ResearchNode, signed, quarantined, challenge_open, research_usable)

After:

(BackwardNode, signed, quarantined, resolved_modify, research_usable)

Meaning:

- review does not erase the object,
- it reclassifies it into a more precise type,
- quarantine remains intact.

10.4 Reopenability Without Execution

Before:

(BackwardNode, signed, quarantined, settled, research_usable)

After:

(BackwardNode, signed, reopenable, settled, research_usable)

Meaning:

- evidence changed enough for reconsideration,
- but not enough for runtime permission.

This is a crucial anti-collapse rule.

11. Transition Guards

Every legal transition must declare one or more guards.

Canonical guard classes:

- `witness_guard`
- `signature_guard`
- `expiry_guard`
- `challenge_guard`
- `review_guard`
- `resource_guard`
- `privilege_guard`
- `time_window_guard`
- `human_anchor_guard`
- `institutional_override_guard`

11.1 Example

quarantined -> reopenable

may require:

- `new_evidence_present = true`
 - `challenge_state in {none, settled}`
 - `expiry_state != expired`
 - `review_guard = satisfied`
-

12. Forbidden Shortcuts (Red Lines)

The matrix must explicitly outlaw the following jumps.

12.1 `cinematic_only -> runtime_usable`

This is the most important anti-laundering rule.

12.2 `asserted -> signed`

No skipping evidence stratification.

12.3 `quarantined -> reopened without reopenable`

No direct escape from quarantine.

12.4 `challenge_open -> archived without outcome`

No quiet burial of unresolved disagreement.

12.5 `expired -> settled`

Expired evidence may remain visible historically but cannot be retroactively stabilized for present use.

12.6 `display_only -> action_authority`

Interface is never a source of runtime authority.

13. Transition Operators

A useful algebraic vocabulary for transitions:

- `observe(x)`
- `witness(x)`
- `sign(x)`
- `expire(x)`
- `challenge(x)`
- `queue(x)`
- `review_uphold(x)`
- `review_modify(x)`
- `review_split(x)`
- `quarantine(x)`
- `mark_reopenable(x)`
- `reopen(x)`
- `archive(x)`
- `seal(x)`

These operators must be:

- total only on valid domains,
- rejected on forbidden inputs,

- witnessable when materially significant.

13.1 Example

`sign(asserted)`

is invalid.

`sign(witnessed)`

may be valid if signature guard is satisfied.

14. Minimal State Machine Example

ExecutionNode

-> observe

Observed Runtime Event

-> witness

GlitchNode[witnessed]

-> quarantine

ResearchNode[witnessed, quarantined]

-> sign

ResearchNode[signed, quarantined]

-> challenge

ResearchNode[signed, quarantined, challenge_open]

-> review_modify

BackwardNode[signed, quarantined, resolved_modify]

-> mark_reopenable

BackwardNode[signed, reopenable, settled]

At no point does this automatically become executable.

That requires a different protocol entirely.

15. Relation to Future Code

This layer is the closest conceptual bridge to implementation.

It suggests future code structures such as:

- enums for each status domain,
- guard-check functions,

- transition reducers,
- forbidden transition assertions,
- witness-trigger hooks,
- challenge-review state machines,
- audit serializers.

In other words:

this is where graph ontology begins to harden into code physiology.

16. Why This Matters

Long-lived systems fail when status is treated as prose.

If the system cannot formally distinguish:

- visible from witnessed,
- witnessed from signed,
- quarantined from reopenable,
- challenge-open from settled,
- research-usable from runtime-usable,

then all higher architecture eventually dissolves into one old failure:

the system acts as though what is merely imaginable is already authorized.

This layer prevents that.

17. Explicit Bridge

This layer turns one architectural truth into lawful mechanics:

a future path may become thinkable long before it becomes admissible.

That is the bridge between:

- SER continuity,
 - L4 bounded reality,
 - and the disciplined refusal to let possibility masquerade as permission.
-

18. Hidden Bridges

Hidden Bridge 1 — Cybernetics

A controller must distinguish internal states clearly enough to regulate transitions.

State ambiguity is control failure.

Hidden Bridge 2 — Information Theory

Signal lineage is preserved only when transitions remain reconstructible.

State algebra preserves transform history instead of flattening it into final labels.

19. Earth Paragraph

In engineering, a component is not “working” simply because it appears on a dashboard. It may be powered off, in test mode, expired, under inspection, or locked out pending review. Serious systems encode those distinctions explicitly, because confusing “visible” with “ready” is how people blow fuses, overheat machines, or trust gauges that were never connected to the circuit. AI status must be treated the same way.

20. Final Position

The State Transition Matrix / Status Algebra Layer is the lawful metabolism of the stack.

It defines:

- what each status means,
- how statuses combine,
- which transitions are legal,
- which jumps are forbidden,
- and how continuity survives reinterpretation without turning into authority leakage.

Without this layer, the graph may still be elegant.

But it will not yet be mechanically sane.